CORRIGÉ TYPE – TP 3

Système de gestion d'une résidence universitaire

Programmation Orientée Objet (POO) en C++ – Analyse, UML et code proposé

Objectif : gérer les demandes d'hébergement, les chambres, les attributions, les départs et les statistiques d'occupation.

Hypothèse : une chambre peut accueillir plusieurs étudiants dans la limite de sa capacité. Une attribution représente une place occupée par un étudiant pendant une période. Le système ne doit jamais dépasser la capacité d'une chambre.

Ce corrigé est une proposition pédagogique complète, à adapter aux conventions de votre enseignant.

1. Analyse du problème

Acteur principal : le responsable de résidence. Il gère les étudiants, les demandes et les chambres. Le système doit garantir la cohérence des affectations et conserver l'historique.

1.1 Règles de gestion

• Chaque étudiant possède un identifiant unique.
• Une résidence contient un ou plusieurs bâtiments ; chaque bâtiment contient plusieurs chambres.
• La capacité d'une chambre est strictement positive.
• Le nombre de places occupées ne dépasse jamais la capacité.
• Une demande concerne un étudiant et possède un statut.
• Une attribution possède un étudiant, une chambre, une date de début et éventuellement une date de fin.
• Un départ clôture l'attribution sans supprimer l'historique.

1.2 Scénario général

Le responsable enregistre une demande, consulte les disponibilités, choisit une chambre et demande l'attribution. Le système vérifie la capacité au dernier moment, crée l'attribution et met la demande à l'état « satisfaite ». Au départ, l'attribution est clôturée et la place est libérée.

2. Diagramme de cas d'utilisation

L'acteur principal est le responsable. Les fonctionnalités attendues sont représentées ci-dessous.

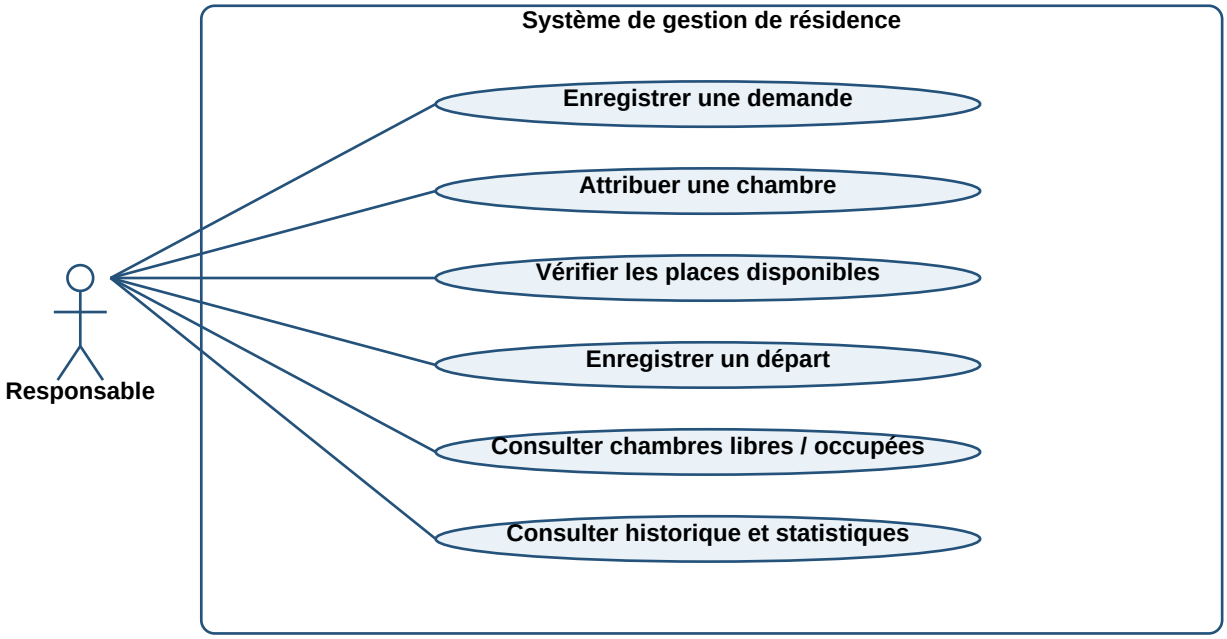

Figure 1 – Diagramme de cas d'utilisation simplifié.

Interprétation : l'attribution inclut obligatoirement une vérification de disponibilité. Le responsable peut aussi consulter les chambres, les occupations, l'historique et les statistiques.

3. Diagramme de classes

Les six classes de l'énoncé sont retenues. Les états sont modélisés par des énumérations afin d'éviter les chaînes de caractères incohérentes.

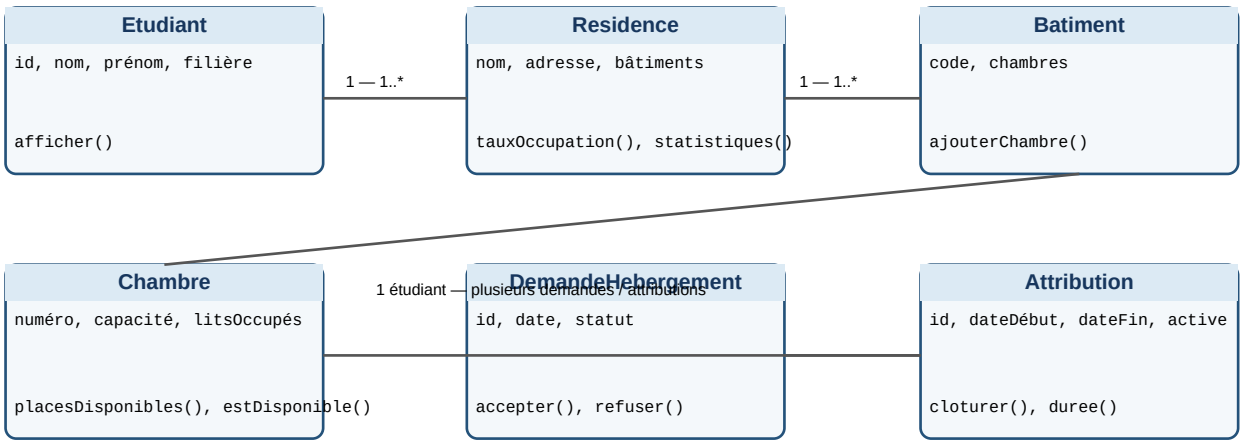

Figure 2 – Diagramme de classes simplifié (attributs et opérations principaux).

Rôle des classes

Etudiant : identifie l'occupant (id, nom, prénom, filière).
Residence : regroupe les bâtiments et calcule les statistiques globales.
Batiment : regroupe les chambres.
Chambre : gère sa capacité et le nombre de places occupées.
DemandeHebergement : conserve la demande et son statut.
Attribution : représente une occupation effective pendant une période.

Relations et multiplicités à retenir

• Residence 1 — 1..* Batiment ; Batiment 1 — 1..* Chambre.
• Etudiant 1 — 0..* DemandeHebergement.
• Etudiant 1 — 0..* Attribution (occupations successives).
• Chambre 1 — 0..* Attribution (historique de la chambre).
• DemandeHebergement 1 — 0..1 Attribution (une demande peut aboutir à une attribution).

4. Diagramme de séquence : attribution

La disponibilité est vérifiée au moment précis de l'attribution. Cette deuxième vérification évite qu'une chambre, disponible lors de la consultation, soit affectée entre-temps.

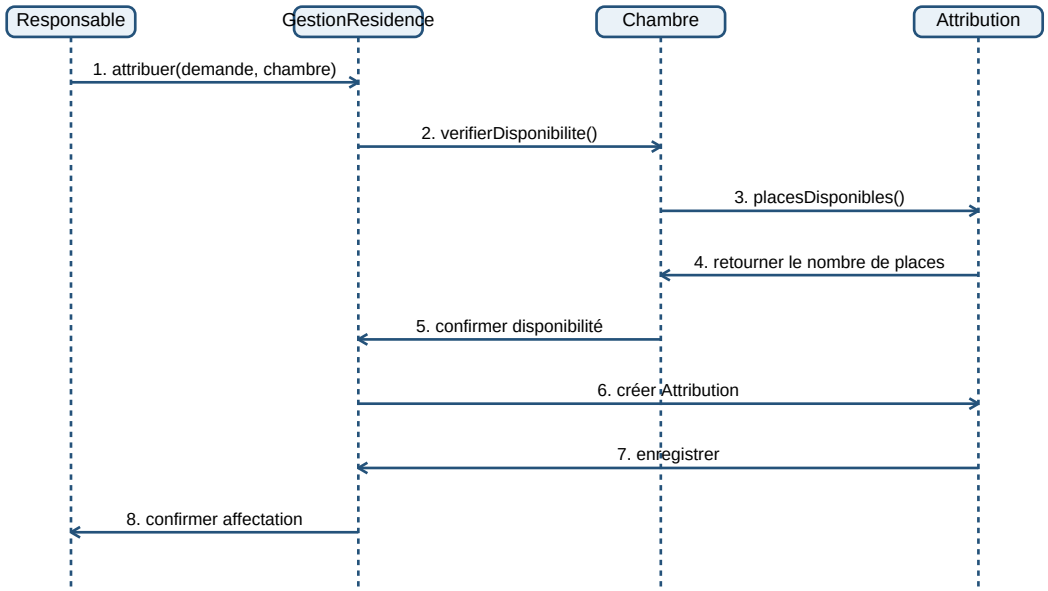

Figure 3 – Séquence simplifiée de l'attribution d'une chambre.

Préconditions : la demande, l'étudiant et la chambre existent ; la demande peut être traitée.
Succès : une attribution active est créée, une place est occupée et la demande devient satisfaite.
Échec : aucune attribution n'est créée si la chambre est complète.

Algorithme

```
attribuer(demande, chambre):
    vérifier la validité de la demande
    vérifier que la chambre existe
    si chambre.placesDisponibles() == 0:
        retourner ECHEC
    créer une Attribution(etudiant, chambre, dateDebut)
    enregistrer l'attribution
    demande.statut = SATISFAITE
    retourner SUCCES
```

5. Diagramme d'états-transitions d'une demande

La demande évolue selon son traitement administratif et le résultat de l'affectation.

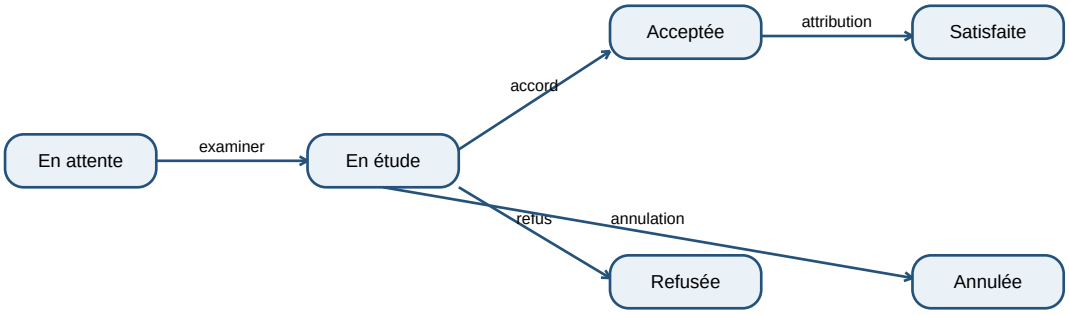

Figure 4 – États et transitions possibles.

Transitions :
• Création → En attente.
• En attente → En étude : examen du dossier.
• En étude → Acceptée : décision favorable.
• En étude → Refusée : décision défavorable.
• Acceptée → Satisfaite : une chambre est réellement attribuée.
• En attente / En étude / Acceptée → Annulée : demande annulée.

Remarque : dans une version simplifiée, l'état « Acceptée » peut être supprimé et la demande passer directement de « En étude » à « Satisfaite » lors de l'attribution.

6. Implémentation en C++

Le code suivant illustre les classes principales et les contrôles essentiels. Les dates sont représentées par des chaînes pour simplifier le TP ; une vraie application utilisera un type Date.

6.1 Classes métier

```
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <stdexcept>
using namespace std;

enum class StatutDemande {
    EN_ATTENTE, EN_ETUDE, ACCEPTEE, REFUSEE, SATISFAITE, ANNULEE
};

class Etudiant {
public:
    int id;
    string nom, prenom, filiere;
    Etudiant(int i, string n, string p, string f)
      : id(i), nom(n), prenom(p), filiere(f) {}
};

class Chambre {
private:
    string numero;
    int capacite, litsOccupes;
public:
    Chambre(string n, int cap) : numero(n), capacite(cap), litsOccupes(0) {
        if (cap <= 0) throw invalid_argument("Capacite invalide");
    }
    int placesDisponibles() const { return capacite - litsOccupes; }
    bool estDisponible() const { return placesDisponibles() > 0; }
    bool occuperUnePlace() {
        if (!estDisponible()) return false;
        ++litsOccupes; return true;
    }
    void libererUnePlace() {
        if (litsOccupes > 0) --litsOccupes;
    }
    string getNumero() const { return numero; }
    int getCapacite() const { return capacite; }
    int getLitsOccupes() const { return litsOccupes; }
};

class DemandeHebergement {
public:
    int id;
    Etudiant* etudiant;
    string dateDemande;
    StatutDemande statut;
    DemandeHebergement(int i, Etudiant* e, string d)
      : id(i), etudiant(e), dateDemande(d),
        statut(StatutDemande::EN_ATTENTE) {}
};

class Attribution {
public:
    int id;
    Etudiant* etudiant;
    Chambre* chambre;
    string dateDebut, dateFin;
    bool active;
    Attribution(int i, Etudiant* e, Chambre* c, string d)
      : id(i), etudiant(e), chambre(c), dateDebut(d),
        dateFin(""), active(true) {}
    void cloturer(string depart) {
        if (!active) return;
        dateFin = depart; active = false;
        chambre->libererUnePlace();
    }
};
```

Les pointeurs Etudiant* et Chambre* référencent des objets créés et gérés ailleurs. Il faut garantir leur durée de vie pendant toute la durée des demandes et attributions. Dans un projet plus avancé, on peut stocker des identifiants ou utiliser des pointeurs intelligents.

6.2 Classe Residence et fonctionnalités

```
class Residence {
private:
    vector<Chambre*> chambres;
    vector<Attribution> historique;
    int prochainId = 1;
public:
    void ajouterChambre(Chambre* c) {
        chambres.push_back(c);
    }

    void afficherDisponibilites() const {
        for (const Chambre* c : chambres)
            cout << "Chambre " << c->getNumero()
                 << " : " << c->placesDisponibles()
                 << " place(s) libre(s)\\n";
    }

    bool attribuer(DemandeHebergement& d, Chambre& c,
                   const string& debut) {
        if (d.etudiant == nullptr ||
            d.statut == StatutDemande::REFUSEE ||
            d.statut == StatutDemande::ANNULEE ||
            d.statut == StatutDemande::SATISFAITE) {
            cout << "Demande non attribuable.\\n";
            return false;
        }
        // Vérification décisive juste avant l'affectation.
        if (!c.occuperUnePlace()) {
            cout << "Echec : chambre complete.\\n";
            return false;
        }
        historique.emplace_back(prochainId++, d.etudiant, &c, debut);
        d.statut = StatutDemande::SATISFAITE;
        cout << "Attribution effectuee.\\n";
        return true;
    }

    bool enregistrerDepart(int id, const string& depart) {
        for (Attribution& a : historique) {
            if (a.id == id && a.active) {
                a.cloturer(depart);
                cout << "Depart enregistre.\\n";
                return true;
            }
        }
        cout << "Attribution active introuvable.\\n";
        return false;
    }

    void afficherHistorique() const {
        for (const Attribution& a : historique)
            cout << "Attribution " << a.id
                 << " | Etudiant : " << a.etudiant->nom
                 << " | Chambre : " << a.chambre->getNumero()
                 << " | Debut : " << a.dateDebut
                 << " | Fin : " << (a.active ? "En cours" : a.dateFin)
                 << "\\n";
    }

    double tauxOccupation() const {
        int total = 0, occupes = 0;
        for (const Chambre* c : chambres) {
            total += c->getCapacite();
            occupes += c->getLitsOccupes();
        }
        if (total == 0) return 0.0;
        return 100.0 * occupes / total;
    }

    void afficherStatistiques() const {
        int total = 0, occupes = 0;
        for (const Chambre* c : chambres) {
            total += c->getCapacite();
            occupes += c->getLitsOccupes();
        }
        cout << "Capacite totale : " << total << "\\n"
             << "Places occupees : " << occupes << "\\n"
             << "Places libres : " << total - occupes << "\\n"
             << fixed << setprecision(2)
             << "Taux d'occupation : " << tauxOccupation() << " %\\n";
    }
};
```

6.3 Programme principal de test

```
int main() {
    Etudiant e1(1, "KOFFI", "Jean", "Informatique");
    Chambre c101("A101", 2);
    Chambre c102("A102", 1);
```

```
    Residence r;
    r.ajouterChambre(&c101);
    r.ajouterChambre(&c102);

    DemandeHebergement d1(1, &e1, "12/11/2026");

    r.afficherDisponibilites();
    r.attribuer(d1, c101, "15/11/2026");
    r.afficherStatistiques();
    r.afficherHistorique();

    r.enregistrerDepart(1, "30/06/2027");
    r.afficherDisponibilites();
    r.afficherStatistiques();
    return 0;
}
```

Compilation : g++ -std=c++11 main.cpp -o residence puis ./residence (Linux/macOS). Sous Windows MinGW : g++ -std=c++11 main.cpp -o residence.exe puis residence.exe.

7. Extension : taux d'occupation et statistiques

Formule :

```
Taux d'occupation = (Places occupées / Capacité totale) × 100
```

Exemple : capacité totale = 120 places ; places occupées = 90. Le taux est (90 / 120) × 100 = **75 %**. Il reste 30 places libres.

Statistiques possibles

• capacité totale ; nombre de places occupées et libres ; taux global ;
• chambres libres, partiellement occupées et complètes ;
• demandes en attente, acceptées, refusées et satisfaites ;
• historique par étudiant, chambre et période ;
• statistiques par bâtiment.

8. Tests de validation

Test	Action	Résultat attendu
T1	Attribuer dans une chambre libre	Attribution créée ; places libres -1
T2	Attribuer dans une chambre complète	Refus ; aucune attribution créée
T3	Enregistrer le départ	Attribution clôturée ; place libérée
T4	Clôturer deux fois la même attribution	Pas de libération supplémentaire
T5	Consulter l'historique après départ	L'ancienne attribution reste visible
T6	Calculer avec capacité totale nulle	Retourne 0 %, pas de division par zéro

9. Conclusion

La solution sépare les responsabilités : Etudiant décrit l'occupant ; Chambre gère sa capacité ; DemandeHebergement suit le traitement administratif ; Attribution conserve les périodes d'occupation ; Residence coordonne les chambres et calcule les statistiques. Le contrôle de disponibilité juste avant l'attribution est essentiel pour éviter les affectations multiples.

Améliorations possibles : sauvegarde dans des fichiers ou une base de données, interface graphique, authentification du responsable, gestion réelle des dates, recherche multicritère et rapports par bâtiment.

Fin du corrigé type – TP 3